WebPage & Deployment

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Practice Guide: Build & Deploy a Static Webpage

Motivation & Context:
In this practice, you will build a personal portfolio website from scratch, learning the fundamentals of HTML, CSS, and deployment. The goal is to understand how modern web pages are structured, styled, and served to the world. By the end, you'll have a site you can personalize and deploy using professional tools.

Install Required Software (Debian/Ubuntu)

We need to set up our Debian (or Debian-based, like Ubuntu) system. We need a text editor, Docker, and its 'compose' plugin.

1. **Update Package Lists:** Open a terminal and run:

   ```
   sudo apt update; sudo apt full-upgrade -y
   sudo apt autoremove -y
   sudo apt autoclean
   ```

2. **Install a Text Editor:** We recommend either Zed or Visual Studio Code for its features. [Students using `WSL` can install Zed/VSCode in Windows and connect it into the VM.]

   ```
   sudo apt install flatpak -y
   flatpak remote-add --if-not-exists flathub \
   https://dl.flathub.org/repo/flathub.flatpakrepo
   flatpak --user install flathub dev.zed.Zed -y
   # OR
   flatpak --user install flathub com.visualstudio.code -y
   ```

Exercise 01: Minimal HTML Page

Goal: Create a basic web page and experiment with HTML and CSS.

1. Create a folder for your project (e.g., `ex01`).
2. Create an `index.html` file with a minimal HTML5 skeleton:
   - Hint: Use `<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`.
   - Add a heading and a short paragraph.
3. Open your page in a browser to verify it works.
4. Add a CSS file (`css/style.css`) and link it in your HTML.
   - Hint: Change the background color and font.
   - Try using an online font (e.g., Google Fonts) via `@font-face` or `<link>`.
5. Experiment with basic styles (colors, fonts, layout).

Exercise 02: Portfolio Skeleton & Media

Goal: Build a portfolio page with semantic structure, media, and deploy it.

1. Create a folder for your portfolio (e.g., `ex02`).
2. Build a portfolio page skeleton:
   - Use semantic HTML: `<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`.
   - Add navigation links to each section.
   - Hint: Use `<figure>` and `<img>` for a profile photo. Try placehold.co for placeholder images.
3. Add sections:
   - About Me: Write a short bio.
   - Projects: List your projects (use `<ul>` or `<article>`).
   - Media: Embed a video and audio: video smaple
     - Video
     - Audio
   - Contact: Add a mailto link.
4. Style your page with CSS for layout, colors, and spacing.
5. Deploy with Docker and Nginx:
   - Create a `compose.yml` file.
   - Use the latest Compose norms.
   - Mount your site folder as a volume.
   - Hint: See Docker Compose documentation.

Exercise 03: Multi-Page Portfolio, Slideshow, Responsive Design, SEO

Goal: Expand your portfolio to multiple pages, add a slideshow, make it responsive, and improve SEO.

1. Use a flat structure (all pages in the same folder: `index.html`, `projects.html`, `contact.html`).
2. Navigation:
   - Add links between pages.
   - Hint: Use `<nav aria-label="Main navigation">`.
3. Project Gallery:
   - Use `<section>` and `<article>` for each project.
   - Add a slideshow using pure HTML/CSS (radio toggles, no JS).
   - Hint: CSS-only slideshow example.
4. About Me:
   - Use `<figure>` and `<img>` for your profile photo.
   - Write a short bio.
5. Media Section:
   - Embed video and audio as in ex02.
6. Contact Page:
   - Add a contact form with fields for name, email, and message.
   - Use `aria-label="Contact form"` for accessibility.
   - Provide a mailto link.
7. Responsive Design (Advanced):
   - **Motivation:** Modern websites must look good on all devices—phones, tablets, and desktops. Responsive design ensures your site adapts to different screen sizes and orientations.
   - **How to achieve it:**
     - Always include the viewport meta tag in your <head>:
       ```
       <meta name="viewport" content="width=device-width, initial-scale=1.0">
       ```
     - Use CSS media queries to change layout and styles for smaller screens:
       ```
       @media (max-width: 600px) {
           header, main, footer {
               padding: 10px;
               margin: 8px;
               max-width: 100%;
           }
           nav a {
               display: block;
               margin: 8px 0;
       ```

```
        }
        img, video {
            max-width: 100%;
            height: auto;
        }
    }
```

- Test your site by resizing your browser window or using device emulation in browser dev tools.

8. SEO Improvements (Advanced):
   - **Motivation:** Search Engine Optimization (SEO) helps your site get discovered by search engines and improves accessibility for users.
   - **How to achieve it:**
     - Add meta tags to your <head> for each page:
       ```
       <meta name="description" content="Describe your page here.">
       <meta name="keywords" content="portfolio, web development, student">
       <meta name="author" content="Your Name">
       ```
     - Use semantic HTML tags (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`) to give meaning to your content.
     - Make sure all images have descriptive `alt` attributes.
     - Use clear, descriptive headings (<h1>, <h2>, etc.).
     - Add `aria-label` attributes to navigation and forms for accessibility:
       ```
       <nav aria-label="Main navigation">
           ...
       </nav>
       <form aria-label="Contact form">
           ...
       </form>
       ```
9. Deploy with Docker and Nginx:
   - Use a `compose.yml` file.
   - Mount your site folder as a volume.
   - Hint: See Docker Compose documentation.

Resources

- MDN HTML Reference
- MDN CSS Reference
- Google Fonts
- Placehold.co
- Docker Compose Docs
- CSS Tricks: CSS-only Carousel
- MDN Responsive Design
- MDN SEO Basics
- MDN Accessibility

Remember:

- Personalize your content and experiment with layout and styles.
- Ask for help or search documentation if you get stuck!